

WSI – Raport z ćwiczenia 4

Zagadnienie przeszukiwania

Karol Zalewski 319128

1. Wprowadzenie

Celem ćwiczenia była implementacja algorytmu gradientu prostego oraz zastosowanie go do znalezienia minimum funkcji Ackleya jedno oraz dwuwymiarowej. Ponadto należało zbadać wpływ rozmiaru kroku na działanie algorytmu. Dodatkowym zadaniem była modyfikacja implementacji, aby algorytm gradientu prostego zawierał także momentum.

2. Założenia projektu

Funkcja Ackleya jedno oraz dwuwymiarowa, która została wykorzystana w zadaniu, wygląda następująco:

$$f(x) = -20 \exp \left(-0.2 \sqrt{x^2} \right) - \exp (\cos(2\pi x)) + 20 + e$$
$$f(x, y) = -20 \exp \left(-0.2 \sqrt{\frac{x^2 + y^2}{2}} \right) - \exp \left(\frac{\cos(2\pi x) + \cos(2\pi y)}{2} \right) + 20 + e$$

Algorytm gradientu prostego z momentum został zaimplementowany w oparciu o analityczny wzór na gradient funkcji Ackleya.

Zastosowany został mechanizm pędu, który pozwalał algorytmowi zapamiętywać prędkość z poprzednich iteracji i unikać zatrzymywania się w małych, lokalnych dołkach funkcji.

Algorytm może obsłużyć dowolną liczbę wymiarów (na potrzeby projektu $n=1$ lub $n=2$, ale n może być w rzeczywistości dowolne) przy użyciu jednej, ogólnej funkcji.

Algorytm kończy działanie, gdy norma gradientu spadnie poniżej ustalonej tolerancji lub gdy zostanie osiągnięty limit iteracji.

3. Prezentacja wyników

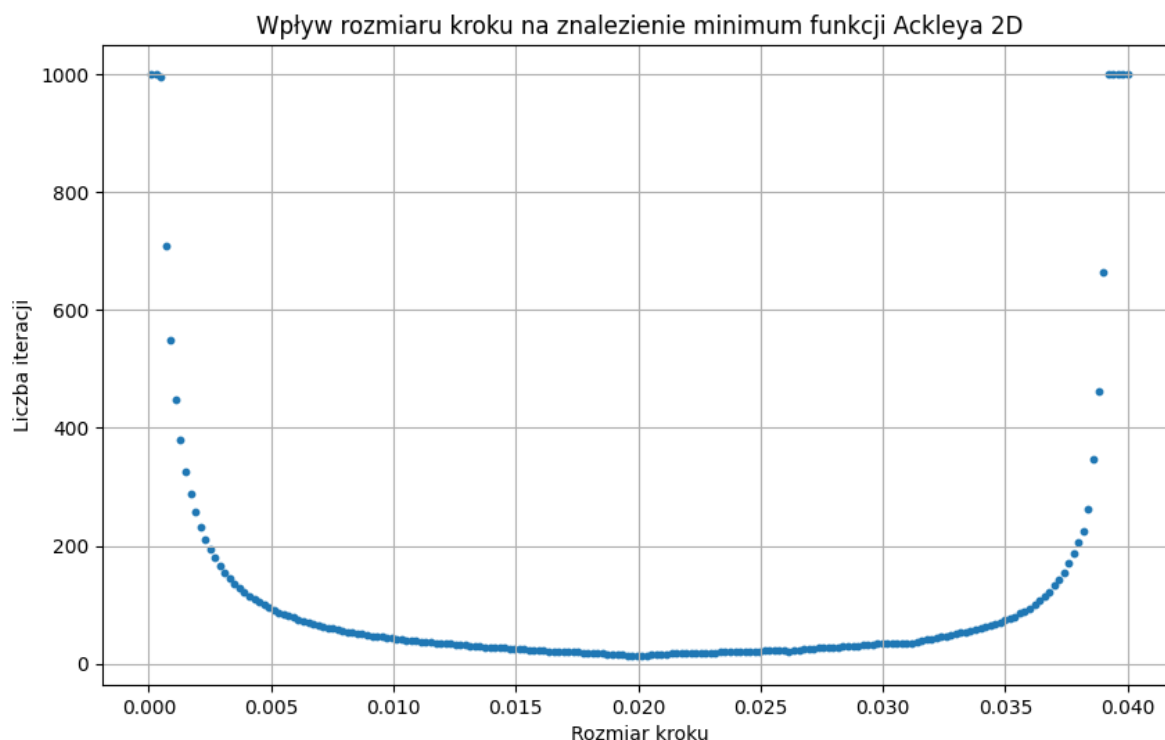
Wstępnie zostały przeprowadzone eksperymenty sprawdzające, ile iteracji jest potrzebnych do znalezienia minimum funkcji w zależności od przyjętego rozmiaru kroku (learning rate). Punkt startowy został przyjęty jako $[2.5]$ lub $[2.5, 2.5]$ odpowiednio dla dwóch różnych wymiarów funkcji, a maksymalną ilość iteracji ustawiono na 1000.

A. Wykres dla funkcji Ackleya jednowymiarowej



Optymalny rozmiar kroku mieści się w zakresie od 0.001 do 0.019.

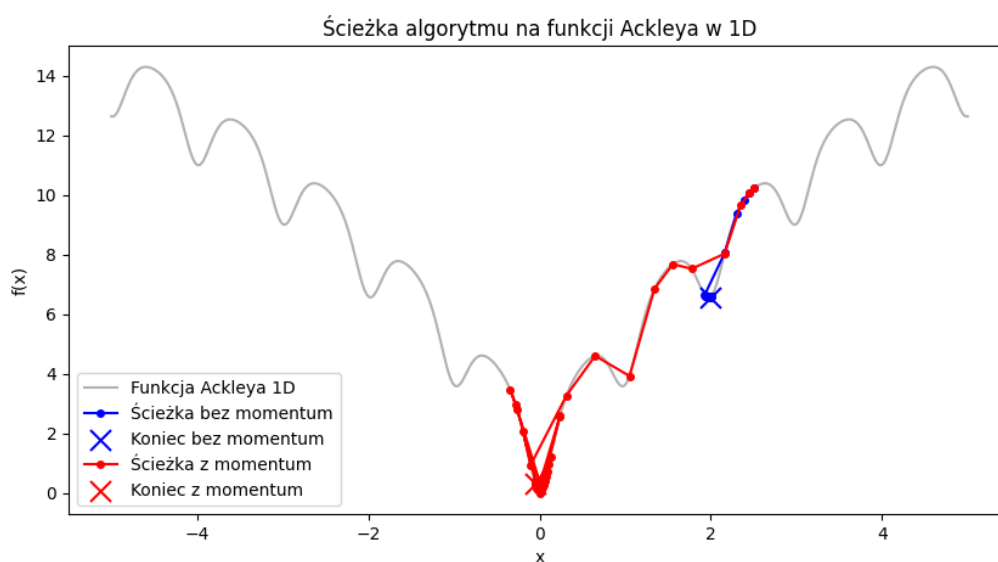
B. Wykres dla funkcji Ackleya dwuwymiarowej



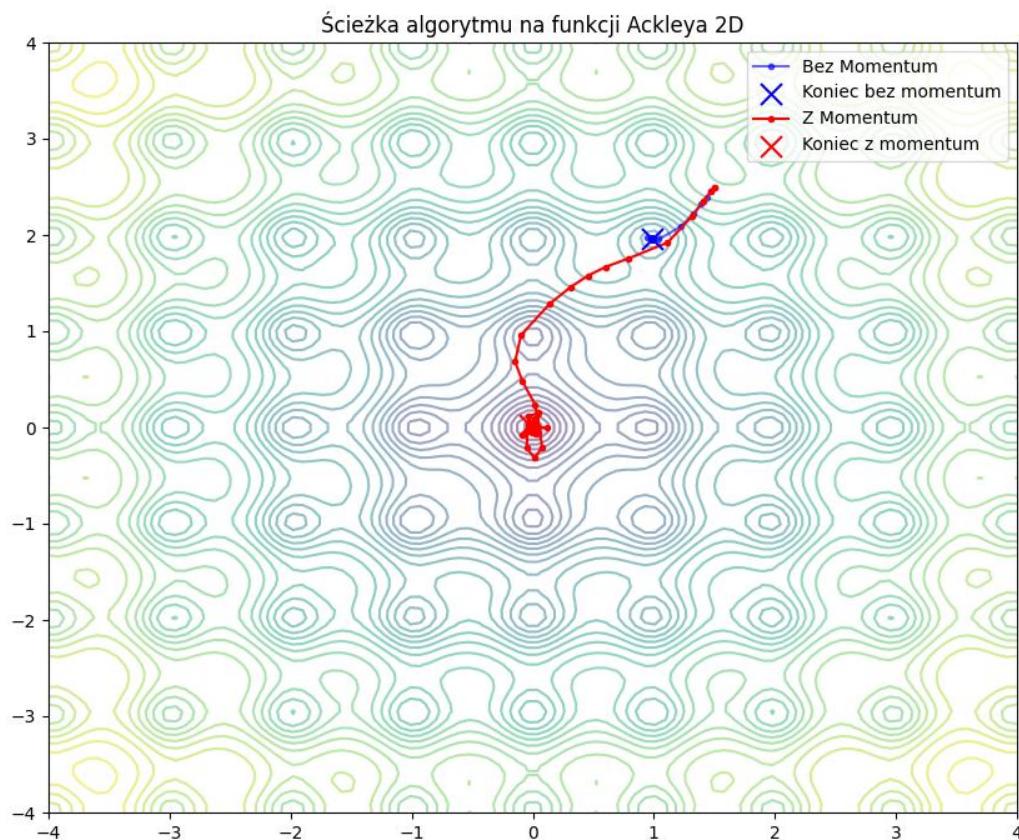
Optymalny rozmiar kroku mieści się w zakresie od 0.002 do 0.038.

Następnie zostało przeprowadzone porównanie działania algorytmu w wariancie z momentum oraz bez momentum (w tym drugim momentum zostało ustawione na 0). W tym celu stworzone zostały wykresy wizualizujące przebieg iteracji w postaci ścieżki (zwykły wykres dla funkcji jednowymiarowej, mapa konturowa dla funkcji dwuwymiarowej).

A. Wykres dla funkcji Ackleya jednowymiarowej (punkt startowy $x=2.5$)

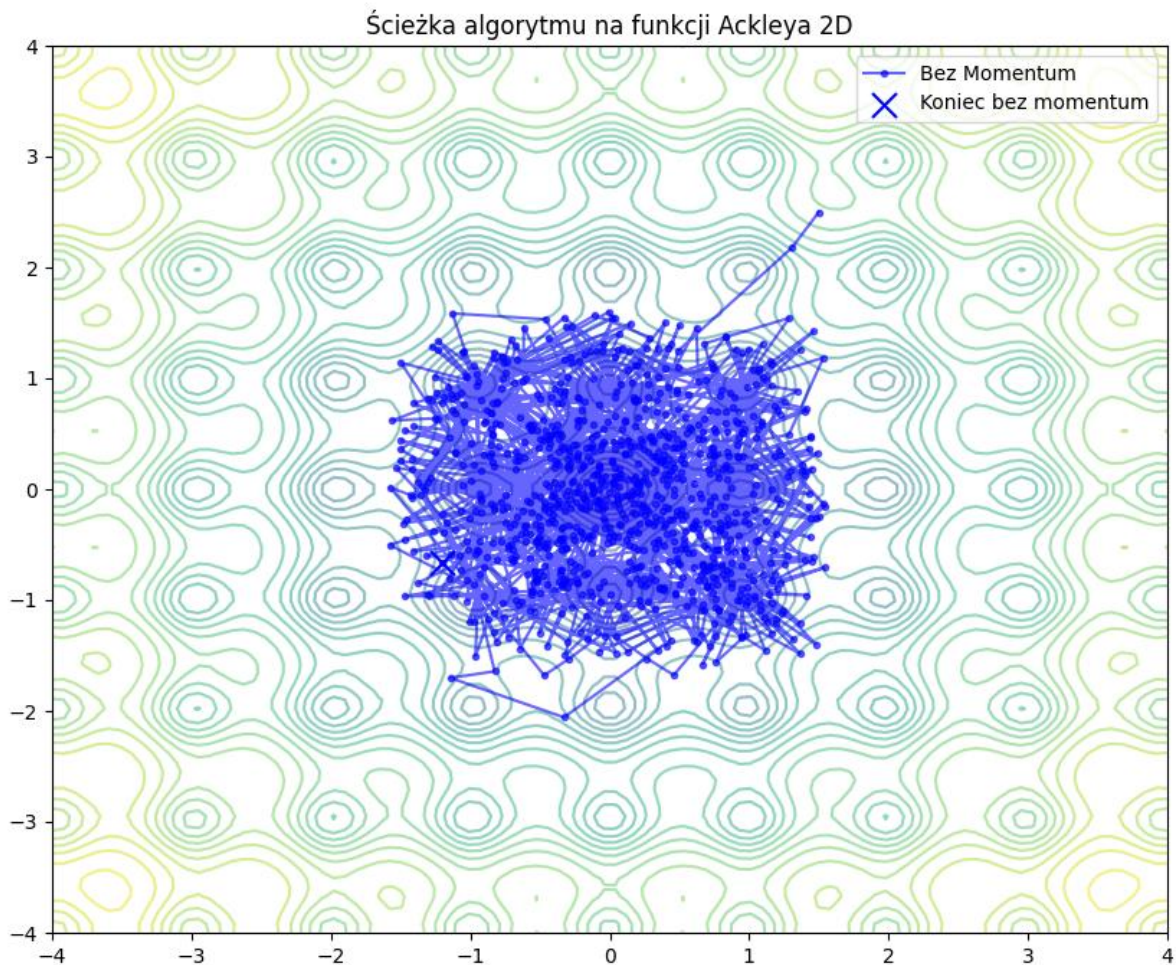


B. Mapa konturowa dla funkcji Ackleya dwuwymiarowej (punkt startowy $(x,y)=(1.5,2.5)$)



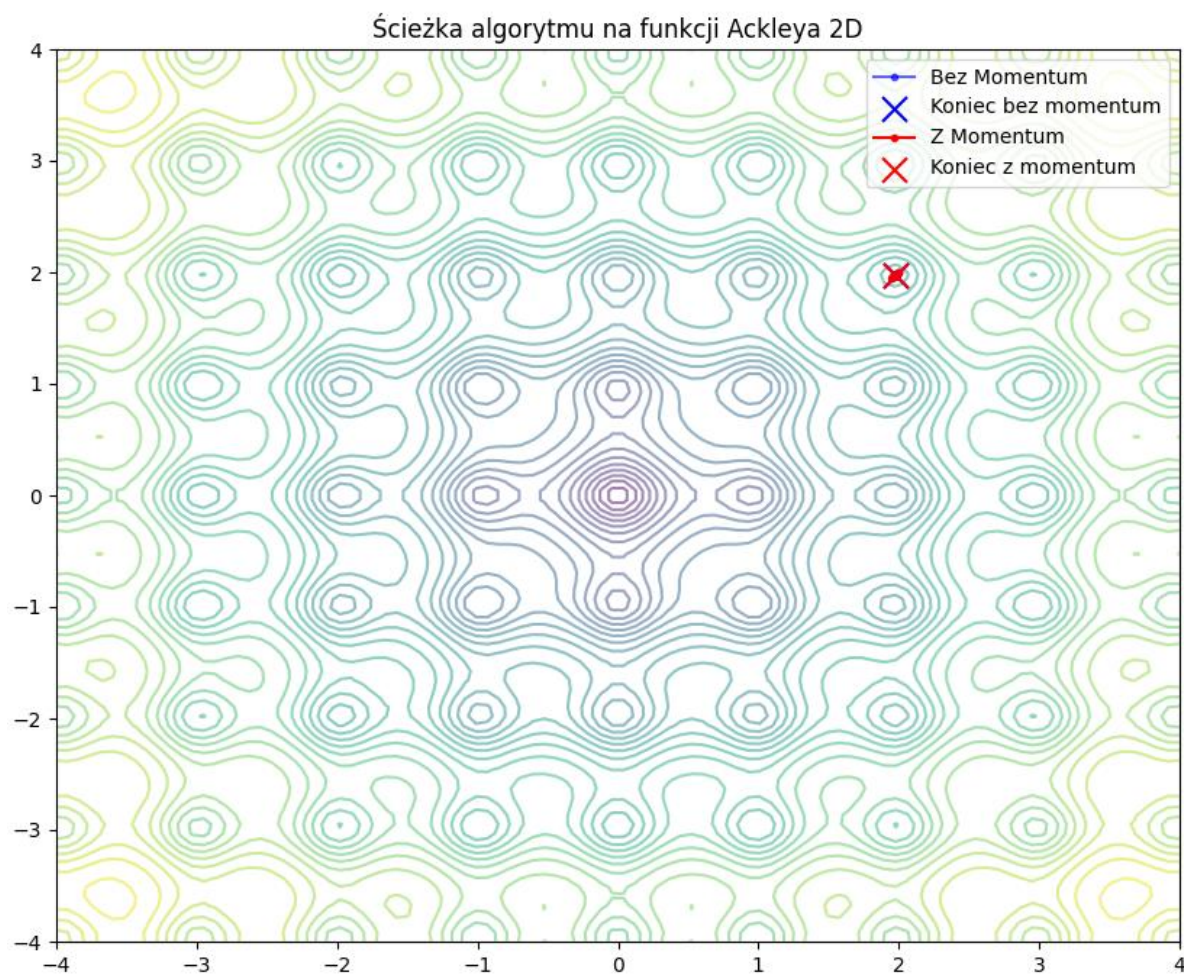
Wyżej przedstawione wykresy pokazują przewagę wariantu z momentum, który daje algorytmowi wystarczająco dużo dodatkowej prędkości, aby uniknął on lokalnych minimów i dostał się do głównego leja funkcji znajdującego się w $[0, 0]$.

Dodatkowo został przeprowadzony test, w którym rozmiar kroku został ustawiony na zbyt dużą wartość (w tym przypadku 0.2):



Jak widać, algorytm nie jest w stanie osiąść w minimum, gdyż co iterację jest zmuszany do poruszenia się zbyt daleko, przez co powstają wyżej przedstawione oscylacje. Istnieje zatem mała, losowa szansa, że z takim rozmiarem kroku uda się osiągnąć pożądane wyniki.

Ostatni test ma na celu sprawdzenie, jak algorytm zachowa się w przypadku ustawienia punktu startowego usytuowanego w jednym z minimów lokalnych. W tym przypadku wybrany został punkt $[2, 2]$. Sprawdzone zostały oba warianty (z momentum oraz bez).



Jak widać, rozpoczęcie w minimum lokalnym powoduje brak możliwości poruszenia się przez algorytm. Jest to spowodowane przez zerowy gradient znajdujący się w minimum, co jest odbierane przez algorytm jako idealnie płaski teren.

4. Wnioski

Zaimplementowany algorytm skutecznie był w stanie znajdować minimum globalne funkcji Ackleya zarówno dla funkcji jednowymiarowej, jak i dwuwymiarowej. Ważne było dobranie odpowiedniego rozmiaru kroku, gdyż zbyt niski nie pozwalał algorytmowi na ucieczkę z lokalnych minimów, a zbyt wysoki powodował niestabilność i przeskakiwanie przez lokalne minima, co nie pozwalało na znalezienie minimum funkcji.

Optymalny rozmiar kroku został ustalony w przedziałach $[0.001, 0.019]$ oraz $[0.002, 0.038]$ odpowiednio dla funkcji jedno oraz dwuwymiarowej. Poza tymi przedziałami algorytm nie był w stanie znaleźć minimum funkcji w rozsądnym czasie (ograniczonym przez maksymalną ilość iteracji).

Wizualizacje w postaci wykresu i mapy konturowej potwierdziły, że zastosowanie mechanizmu momentum wpływało pozytywnie na skuteczność algorytmu. Zapamiętywanie przez algorytm prędkości z poprzednich iteracji ułatwiało poruszanie się w odpowiednim kierunku, a także zapobiegało utknięciu w lokalnym minimum.